

Prefer `std::ranges::contains` over `std::basic_string_view::contains`

Document #: P2302R0
Date: 2021-02-10
Project: Programming Language C++
Audience: Library Evolution Working Group (LEWG)
Reply-to: Christopher Di Bella
<cjdb.ns@gmail.com>

Contents

- [1 Abstract](#)
- [2 Motivation](#)
- [3 Design](#)
 - [3.1 API](#)
- [4 Changes to LEWG Policy Standing Document](#)
- [5 Blunt proposal](#)
 - [5.1 Proposed wording](#)
- [6 Acknowledgements](#)
- [7 References](#)

§ 1 Abstract

[[P1679R3](#)] paves the way for both `std::basic_string::contains` and `std::basic_string_view::contains` to be standardised in C++23, allowing for users to conveniently check whether or not a string contains a specific character or substring of characters. The author of P2302 raised concerns about LEWG introducing member functions to a particular container in [[P1659R1](#)], and seeks to repeal P1679 in favour of an algorithm that achieves the same effort.

§ 2 Motivation

As in P1659, the author is of the opinion that a member-wise operations are the wrong abstraction in C++, when an equivalent algorithm is available. For convenience, and

because the motivation for P2302 is literally identical, it has been copied here below, with alterations to accommodate the current situation.

C++23 introduces `basic_string_view::contains`. Both `basic_string` and `basic_string_view` are peculiar container-like types, with many member functions that duplicate algorithm functionality with minor interface changes (e.g. `compare`, `copy`, `find`, `rfind`, `find_first_of`, etc.). P1679. P1679 §4.2 notes that the decision to add `contains` as a member function was because P1679 agrees with [N3609] in that `contains(haystack, needle)` is ambiguous with `contains(needle, haystack)`.

Although there is prior art with respect to `basic_string`'s member functions, the author expresses concern that our string types have a large set of member functions that either duplicate the algorithms or are directly incompatible with them, and thus limit the amount of composition that's possible. Templates are one of C++'s greatest strengths, and with iterators, we have an extremely extensible and powerful generic programming model. We should take advantage of this model wherever possible to ensure that we do not paint ourselves into a corner with a single type.

At present, it isn't immediately possible to query whether or not any range — other than a C++23 standard string type — contains an element.

It is interesting to note that, since one of the overloads in the `contains` overload set can be implemented search, we are able to query more than just “does haystack contain needle?”: we're also able to query “does haystack contain something that resembles needle?”. See below for examples. This is something that the string-based member functions are unable to do, although the author acknowledges that P1679 mentions case-insensitivity was discussed, and that when framed as a string-only operation, it would be irresponsible to incorporate into the design. By making `contains` an algorithm, we can sidestep this issue altogether, and the post-text revolution simply needs to expose codepoint-relevant operations.

Concerns were outlined in all of N3609, P0457, P1679 about the ambiguity of whether we are performing `contains(haystack, needle)` or `contains(needle, haystack)`. There is prior art in the algorithms library that makes the first range the subject of the operation: `mismatch`, `equal`, `search`, `find_first_of`, and `lexicographical_compare` all take the form `algorithm(haystack, needle)`, so the author remains unconvinced about the ambiguity. LEWG approved P1659 in the Cologne 2019 meeting: meaning that this working group agrees that there isn't any ambiguity, or that LEWG is inconsistent in what it agrees upon.

The author agrees with the authors of P1679 in principle: there should be algorithms that check for these kinds of functionality. Where we disagree is on their placement. Since this functionality is likely useful for vectors, deques, arrays, static vectors, small vectors,

ropes, circular buffers, and an infinite subset of composable ranges, the author strongly argues that `contains` should be an algorithm, not a member function to be added to every type that needs it. Further evidence for the existence of a `contains` algorithm is that the STL gave C++98 a poorly-named `contains` algorithm called `binary_search`, and `any_of` is “contains with a predicate”, showing that there’s prior art for this `contains` to be an algorithm.

§ 3 Design

The proposed usage looks like this:

```
if (std::ranges::contains(haystack, 'o')) {  
    // meow  
}  
  
if (std::ranges::contains(haystack.begin(), haystack.end(), 'c')) {  
    // purr  
}  
  
if (std::ranges::contains(haystack.begin(), haystack.end(),  
    long_needle.begin(), long_needle.end())) {  
    // hiss  
}  
  
if (std::ranges::contains(haystack, long_needle)) {  
    // hiss again  
}  
  
if (std::ranges::contains(haystack, long_needle, bind_back(std::modulo(),  
    4))) {  
    // double purr  
}
```

As this is an algorithm, the author suggests this be placed in namespace `std::ranges`, where new algorithms are currently destined to go. As usual with new algorithms, these ones will also have projections.

§ 3.1 API

The author notes that for an unsorted sequence, a `contains` algorithm is simply one of the following:

```
auto const haystack = "the quick brown fox jumps over the lazy dog"sv;  
  
stdr::find(haystack, 'o') != haystack.end(); // haystack.contains('o')  
not stdr::search(haystack, "red"sv).empty(); // haystack.contains("red"sv)  
  
// See https://godbolt.org/z/vTendW
```

As such, `contains` is a wrapper around these two algorithms:

```

namespace std {
    template<input_iterator I, sentinel_for<I> S, class T, class Proj =
        identity>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>,
        const T*>
    constexpr bool ranges::contains(I first, S last, const T& value, Proj
        proj = {});

    template<input_range R, class T, class Proj = identity>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    constexpr bool ranges::contains(R&& r, const T& value, Proj proj = {});

    template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
        sentinel_for<I2> S2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool
        ranges::contains(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred
            = {},
                        Proj1 proj1 = {}, Proj2 proj2 = {});

    template<forward_range R1, forward_range R2, class Pred =
        ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
    constexpr bool
        ranges::contains(R1&& r1, R2&& r2, Pred pred = {},
                        Proj1 proj1 = {}, Proj2 proj2 = {});
}

```

As previously noted, a predicate-based “contains_if” already exists as any_of.

§ 4 Changes to LEWG Policy Standing Document

P1659 notes:

In response to the concerns outlined in the motivation section of this document, the author would like to request that LEWG consider discussing adopting in SD-8, a policy of ensuring that options for algorithms are preferred when a proposal to add a member function to a container-like type is considered.

P1659 was discussed in the Cologne 2019 meeting, where it was noted that the requested change to SD-8 would be better placed in the LEWG Policy Standing Document. There was agreement for this document to be updated to reflect this, but the approval of P1679 indicates that it might have slipped through the cracks.

The author would like to request that the working group confirm the presence of this rule, and add it if it is missing. The problem was frustrating enough when we only needed to

worry about standard containers, but is now exacerbated thanks to the standard library’s acknowledgement of ranges.

Since the concern about “ambiguity” regarding haystacks and needles tends to pop up on occasion, it would be worth considering the addition of some additional wording to make it clear this problem has been a non-issue since the very beginning of standard C++.

§ 5 Blunt proposal

Remove both `std::basic_string::contains` and `std::basic_string_view::contains`, and to add the following proposed wording to the algorithms clause.

§ 5.1 Proposed wording

Remove the wording added by P1679R3.

Add the following text to 25.4 [\[algorithm.syn\]](#):

```
// [alg.none.of], none of
// ...

// [alg.contains], contains
template<input_iterator I, sentinel_for<I> S, class T, class Proj =
    identity>
    requires indirect_binary_predicate<ranges::equal_to, projected<I,
        Proj>, const T*>
    constexpr bool ranges::contains(I first, S last, const T& value, Proj
        proj = {});
template<input_range R, class T, class Proj = identity>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    constexpr bool ranges::contains(R&& r, const T& value, Proj proj =
        {});

template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
    sentinel_for<I2> S2, class Pred = ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool ranges::contains(I1 first1, S1 last1, I2 first2, S2
        last2,
                                Pred pred = {}, Proj1 proj1 = {},
                                Proj2 proj2 = {});
template<forward_range R1, forward_range R2, class Pred =
    ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
    constexpr bool ranges::contains(R1&& r1, R2&& r2, Pred pred = {},
                                Proj1 proj1 = {}, Proj2 proj2 = {});

// [alg.foreach], for each
```

Add the following to 25.6 [alg.nonmodifying]:

```
template<input_iterator I, sentinel_for<I> S, class T, class Proj =
    identity>
    requires indirect_binary_predicate<ranges::equal_to, projected<I, Proj>,
        const T*>
    constexpr bool ranges::contains(I first, S last, const T& value, Proj
        proj = {});

template<input_range R, class T, class Proj = identity>
    requires indirect_binary_predicate<ranges::equal_to,
        projected<iterator_t<R>, Proj>, const T*>
    constexpr bool ranges::contains(R&& r, const T& value, Proj proj = {});
```

Returns: `ranges::find(first, last, value, proj) != last`.

```
template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
    sentinel_for<I2> S2, class Pred = ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
    constexpr bool ranges::contains(I1 first1, S1 last1, I2 first2, S2
        last2,
                                Pred pred = {}, Proj1 proj1 = {}, Proj2
        proj2 = {});

template<forward_range R1, forward_range R2, class Pred =
    ranges::equal_to,
    class Proj1 = identity, class Proj2 = identity>
    requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred,
        Proj1, Proj2>
    constexpr bool ranges::contains(R1&& r1, R2&& r2, Pred pred = {},
        Proj1 proj1 = {}, Proj2 proj2 = {});
```

Returns: `return ranges::search(first1, last1, first2, last2, pred, proj1, proj2).empty() == false`.

§ 6 Acknowledgements

The author would like to thank Corentin Jabot for reviewing this proposal.

§ 7 References

[N3609] Jeffrey Yasskin. 2013-03-15. `string_view`: a non-owning reference to a string, revision 3.

<https://wg21.link/n3609>

[P1659R1] Christopher Di Bella. 2020-07-15. `starts_with` and `ends_with`.

<https://wg21.link/p1659r1>

[P1679R3] Wim Leflere, Paul Fee. 2020-07-22. `String Contains` function.

<https://wg21.link/p1679r3>